ENGR 102 Summer Bridge
Day 13 Instructor Key

Loop Design, Continue, Break, and Nested State

20 points • approximately 20-25 minutes

Name		UIN		Section	

Boolean-operator convention. Python operators appear as `and`, `or`, and `not`. Their lowercase spelling is required in code.

Show intermediate state for trace credit. Program credit requires one coherent solution. Unless a prompt says otherwise, give exact Python 3 output.

Question 1. Trace nested skip, stop, reset, and accumulation. 5 points

```
overall = 0

for row in range(2):
    row_total = 0
    for value in [0, row + 2, 5]:
        if value == 0:
            continue
        if value == 5:
            break
        row_total = row_total + value
    overall = overall + row_total
    print(row, row_total)

print(overall)
```

Give a reached-pass ledger and explain exactly which loop break ends.

Answer and explanation

Row 0 resets row_total, skips 0, adds 2, and breaks at 5; it prints 0 2. Row 1 resets, skips 0, adds 3, and breaks at 5; it prints 1 3. overall becomes 5 and prints 5. break ends only the inner loop.

Question 2. Repair an accumulation/reset defect. 3 points

```
hits = 0

for row in range(2):
    row_total = 0
    for value in [1, 2]:
        row_total = row_total + value
    hits = row_total

print(hits)
```

The intent is to accumulate both row totals. Give actual, intended, and the minimal repair.

Answer and explanation

Each row_total is 3, but hits = row_total overwrites the earlier total, so actual output is 3. The intended total is 6. Repair with hits = hits + row_total after each inner loop.

Question 3. Program construction — 12 points

Write one complete Python program that satisfies every requirement.

- Attempt to read up to six integers using a for loop.
- Stop immediately when a negative value is read.
- Skip zero without adding it or counting it.
- For positive values, accumulate total and accepted_count.
- Print Total: and Accepted: on separate lines. Include a test with both skip and stop behavior.

Answer and explanation

```
total = 0
accepted_count = 0

for attempt in range(6):
    value = int(input())
    if value < 0:
        break
    if value == 0:
        continue
    total = total + value
    accepted_count = accepted_count + 1

print(f"Total: {total}")
print(f"Accepted: {accepted_count}")
```

The negative test must occur before the zero and accumulation work because it ends repetition. continue skips the remaining body for zero but allows the next attempt.

Inputs 3, 0, 4, -1 produce Total: 7 and Accepted: 2 without requesting two more values. Six positive values 1 through 6 produce Total: 21 and Accepted: 6.

Scoring guidance

2 initialization/range, 2 stop placement, 2 skip placement, 3 accumulation/count, 1 exact output, 2 tests/explanation.